

Inhalt

1 CC ADR Stoch Grid Trading Robot - Technische Dokumentation	1
2 Prompt:	26
3 Kursbeschreibung für den prompt.....	27
4) Einbaue fonic news	43
5 Kurzbeschreibung für Menschen	43

1 CC ADR Stoch Grid Trading Robot- Technische Dokumentation

Version: 1.28

Copyright: Algorithm Factory

Dateiname: CC_ADR_Stoch_Grid_with_Buttons.mq5

Plattform: MetaTrader 5



Inhaltsverzeichnis

1. [Übersicht](#)
2. [Architektur & Code-Struktur](#)
3. [Trading-Logik](#)
4. [Parameter-Referenz](#)
5. [Funktions-Referenz](#)
6. [Ablaufdiagramme](#)
7. [GUI-Steuerung](#)
8. [Risiko-Management](#)



Übersicht

Konzept

Der EA ist ein **Mean-Reversion Grid Trading System**, das auf überdehnte Marktbewegungen setzt und durch Averaging-Down profitiert. Der Robot nutzt:

- **ADR (Average Daily Range)** zur Messung von Markt-Überdehnung
- **RSI (Relative Strength Index)** als alternative Entry-Bedingung
- **Stochastik** für präzises Entry-Timing
- **Grid-System** zum Averaging bei gegenlaufenden Bewegungen
- **Dynamische Targets** die sich mit zunehmender Grid-Tiefe reduzieren

Haupt-Features

- ✓ Zwei Entry-Modi: ADR oder RSI
- ✓ Stochastik-Filter für Timing-Optimierung
- ✓ Dynamisches Grid-System mit Averaging
- ✓ Money Management mit Balance-Skalierung
- ✓ GUI mit 4 Buttons zur Live-Steuerung
- ✓ Multi-Symbol-fähig durch Magic Numbers
- ✓ Individueller SL pro Trade
- ✓ Dynamische Exit-Targets (degressive Ziele)
- ✓ End-of-Day Close-Funktion
- ✓ Echtzeit-Monitoring von MaxDD

Architektur & Code-Struktur

Programmierparadigma

Prozedural - Keine OOP-Klassen, alles als globale Funktionen implementiert.

Include-Dependencies

```
#include <Trade\Trade.mqh> // Standard MQL5 Trade-Klasse
```

Enumerationen

ENUM_ENTRY_MODE

Bestimmt welche Logik für den ersten Entry genutzt wird:

```
enum ENUM_ENTRY_MODE
{
    MODE_ADR = 0, // ADR Entry: Wartet bis Tagesrange > X% von ADR
    MODE_RSI = 1 // RSI Entry: Wartet auf RSI-Extreme (D1)
};
```

ENUM_DIR_FILTER

Runtime-Filter für erlaubte Handelsrichtungen:

```
enum ENUM_DIR_FILTER
{
```

```

FILTER_BOTH   = 0, // Long & Short erlaubt
FILTER_LONG   = 1, // Nur Long
FILTER_SHORT  = 2, // Nur Short
FILTER_NEUTRAL = 3 // Keine Trades (Pause)
};

```

Globale Variablen

Trading-Objekte

```

CTrade trade;           // Trade-Execution-Objekt
int handleStoch;         // Stochastik-Indikator Handle (H1)
int handleRSI = INVALID_HANDLE; // RSI-Indikator Handle (D1, optional)

```

ADR & Range

```

double adrValue = 0.0;      // Aktueller ADR-Wert in Points
double dayRange = 0.0;      // Aktuelle Tagesrange (High - Low)
datetime lastProcessedH1Bar = 0; // Verhindert Mehrfach-Processing derselben H1-Kerze

```

Statistik & Monitoring

```

double stat_HighWaterMark = 0.0; // Höchste Balance (Referenz für DD)
double stat_MaxDD_Equity_Money = 0.0; // Max DD in Geldeinheiten
double stat_MaxDD_Equity_Percent = 0.0; // Max DD in Prozent
string stat_EA_Action = "Initializing..."; // Aktueller Status-Text

```

Runtime State

```

ENUM_DIR_FILTER g_currentFilter; // Aktueller Richtungs-Filter (änderbar via GUI)

```

GUI Object Names

```

// Buttons
string btnBothName  = "Btn_Dir_Both";
string btnLongName  = "Btn_Dir_Long";
string btnShortName = "Btn_Dir_Short";
string btnNeutralName = "Btn_Dir_Neutral";

// Labels
string lblMaxDDName = "Lbl_Info_MaxDD";
string lblPosName   = "Lbl_Info_Pos";

```

```
string lblStatusName = "Lbl_Info_Status";
```

```
string lblSetupName = "Lbl_Info_Setup";
```

Trading-Logik

Logik-Fluss (OnTick)

OnTick() wird bei jedem neuen Tick aufgerufen

```
|  
|  
| └─► 1. MONITORING  
|   |  
|   | └─ Balance prüfen  
|   |  
|   | └─ High Water Mark aktualisieren  
|   |  
|   | └─ Max Drawdown berechnen  
|   |  
|   |  
|   | └─► 2. PREPARATIONS  
|   |  
|   | └─ ValidatePositionsAgainstFilter() - Schließt ungültige Positionen  
|   |  
|   | └─ CalculateADR_and_Range() - Berechnet ADR & aktuelle Range  
|   |  
|   | └─ Zähle offene Positionen (nur eigene Magic Number)  
|   |  
|   |  
|   | └─► 3. STATUS LOGIC  
|   |  
|   | └─ Setze stat_EA_Action Text  
|   |  
|   |  
|   | └─► 4. LOGIC EXECUTION  
|   |  
|   | |  
|   | | └─► Wenn Positionen > 0:  
|   | |  
|   | | | └─ CheckBasketExit() - Prüfe Exit-Bedingung  
|   | | |  
|   | | | └─ Bei neuer H1-Kerze: Grid-Nachkauf wenn Distanz erreicht  
|   | | |  
|   | | |  
|   | | | └─► Wenn EOD-Zeit erreicht:  
|   | | |  
|   | | | └─ CloseAllPositions()  
|   | | |  
|   | | |  
|   | | | └─► Wenn keine Positionen & neue H1-Kerze:  
|   | | |  
|   | | | |
```

```

|   |─► Entry-Bedingung prüfen (ADR ODER RSI)
|   |   |─ MODE_ADR: dayRange > (ADR * InpEntryADR_Pct / 100)
|   |   |─ MODE_RSI: RSI >= Upper ODER RSI <= Lower
|   |
|   |─► Wenn Entry-Bedingung erfüllt:
|       |─ Stochastik-Crossover prüfen
|           |─ Short Entry: K kreuzt UNTER InpStochUpper
|           |─ Long Entry: K kreuzt ÜBER InpStochLower
|
|─► 5. VISUALS UPDATE
    |─ UpdateVisuals() - Aktualisiere GUI-Labels

```

Entry-Logik im Detail

Schritt 1: Entry-Modus wählen (ADR oder RSI)

MODE_ADR:

Bedingung: $\text{dayRange} > (\text{adrValue} * \text{InpEntryADR_Pct} / 100.0)$

Beispiel:

- ADR = 100 Points
- InpEntryADR_Pct = 150%
- Trigger = 150 Points
- Wenn aktuelle Tagesrange > 150 Points → Entry erlaubt

MODE_RSI:

Bedingung: $\text{RSI}[0] \geq \text{InpRSI_Upper}$ ODER $\text{RSI}[0] \leq \text{InpRSI_Lower}$

Beispiel:

- InpRSI_Upper = 70
- InpRSI_Lower = 30
- Wenn $\text{RSI} > 70$ oder $\text{RSI} < 30$ → Entry erlaubt

Schritt 2: Stochastik-Timing

Erst wenn Entry-Bedingung (ADR/RSI) erfüllt ist:

Short Entry:

Bedingung: $k[2] \geq \text{InpStochUpper}$ UND $k[1] < \text{InpStochUpper}$

Bedeutet: Stochastik war überkauft und kreuzt zurück

Long Entry:

Bedingung: $k[2] \leq \text{InpStochLower}$ UND $k[1] > \text{InpStochLower}$

Bedeutet: Stochastik war überverkauft und kreuzt zurück

Grid-Logik

Grid-Nachkauf-Bedingungen

1. **Basket bereits offen** ($\text{openPositions} > 0$)
2. **Noch nicht Max-Positionen erreicht** ($\text{openPositions} < \text{InpMaxPositions}$)
3. **Neue H1-Kerze** (verhindert Spam)
4. **Preis hat sich um Grid-Distanz gegen Position bewegt**
5. **Richtungs-Filter erlaubt Grid** (z.B. kein Grid für Shorts wenn Filter = LONG)

Grid-Distanz-Berechnung

$\text{requiredDist} = \text{adrValue} * (\text{InpGridStep_ADR_Pct} / 100.0)$

Beispiel:

- ADR = 100 Points

- InpGridStep_ADR_Pct = 25%

→ Grid-Abstand = 25 Points

Grid-Entry-Logik

Short Grid:

Wenn: $\text{ask} \geq (\text{lastEntryPrice} + \text{requiredDist})$

→ Verkaufe Grid-Lot bei aktuellem Ask

→ $\text{SL} = \text{Ask} + \text{slDistancePoints}$

Long Grid:

Wenn: $\text{bid} \leq (\text{lastEntryPrice} - \text{requiredDist})$

→ Kaufe Grid-Lot bei aktuellem Bid

→ $\text{SL} = \text{Bid} - \text{slDistancePoints}$

Exit-Logik

Basket-Exit (CheckBasketExit)

Berechnung des durchschnittlichen Entry-Preises:

$\text{avgPrice} = \Sigma(\text{Preis} * \text{Volumen}) / \Sigma(\text{Volumen})$

Dynamisches Target:

$\text{targetPct} = \text{InpStartTarget_ADR} - ((\text{count} - 1) * \text{InpTargetDecay_ADR})$

$\text{targetDistPoints} = \text{adrValue} * (\text{targetPct} / 100.0)$

Beispiel:

- Position 1: Target = 10% ADR
- Position 2: Target = 5% ADR (10% - 1*5%)
- Position 3: Target = 0% ADR (10% - 2*5%) → Breakeven

Exit-Trigger:

Long Basket: $\text{currentBid} \geq (\text{avgPrice} + \text{targetDistPoints})$

Short Basket: $\text{currentAsk} \leq (\text{avgPrice} - \text{targetDistPoints})$

Individueller Stop Loss

Jede Position bekommt einen eigenen SL:

$\text{slDistancePoints} = \text{adrValue} * \text{InpIndividualSL_ADR}$

Beispiel:

- ADR = 100 Points
- InpIndividualSL_ADR = 3.0
- SL = 300 Points Abstand vom Entry

Parameter-Referenz

Direction Filter

Parameter	Typ	Default	Beschreibung
InpStartDirection	ENUM_DIR_FILTER	FILTER_BOTH	Start-Richtung beim EA-Start

Positions Management

Parameter	Typ	Default	Beschreibung
InpFirstLot	double	0.02	Lot-Größe für ersten Entry

Parameter	Typ	Default	Beschreibung
InpGridLot	double	0.01	Lot-Größe für Grid-Nachkäufe
InpMaxPositions	int	10	Maximale Anzahl Grid-Levels
InpIndividualSL_ADR	double	3.0	Stop Loss pro Trade (Vielfaches von ADR)

Money Management

Parameter	Typ	Default	Beschreibung
InpUseMM	bool	false	MM aktivieren (Balance-Skalierung)
InpRefBalance	double	10000.0	Referenz-Balance für MM
InpBalanceStep_Pct	double	20.0	Ab welchem Profit-% skaliert wird
InpLotIncrease_Pct	double	20.0	Um wieviel % die Lots erhöht werden

MM-Beispiel:

Ref-Balance: 10.000€

Balance-Step: 20%

Lot-Increase: 20%

Bei 12.000€ Balance (+20%) → Lots * 1.2

Bei 14.000€ Balance (+40%) → Lots * 1.4

Bei 16.000€ Balance (+60%) → Lots * 1.6

Entry Logic Selection

Parameter	Typ	Default	Beschreibung
InpEntryMode	ENUM_ENTRY_MODE	MODE_ADR	ADR oder RSI Entry

ADR Settings

Parameter	Typ	Default	Beschreibung
InpADRPPeriod	int	14	Anzahl Tage für ADR-Berechnung
InpEntryADR_Pct	double	150.0	Wieviele % von ADR für Entry nötig
InpGridStep_ADR_Pct	double	25.0	Grid-Abstand in % von ADR

RSI Settings (D1)

Parameter Typ Default Beschreibung

InpRSI_Period	int	14	RSI-Periode
InpRSI_Upper	int	70	RSI obere Grenze
InpRSI_Lower	int	30	RSI untere Grenze

Dynamic Exit Targets

Parameter Typ Default Beschreibung

InpStartTarget_ADR	double	10.0	Target bei erster Position (% von ADR)
InpTargetDecay_ADR	double	5.0	Target-Reduktion pro Grid-Level (% von ADR)

Stochastic Settings (H1)

Parameter Typ Default Beschreibung

InpStochK	int	5	K-Periode
InpStochD	int	3	D-Periode
InpStochSlowing	int	3	Slowing
InpStochUpper	int	80	Obere Grenze (Überkauft)
InpStochLower	int	20	Untere Grenze (Überverkauft)

Time Filter

Parameter Typ Default Beschreibung

InpUseEODClose	bool	false	EOD-Close aktivieren
InpEODHour	int	23	Stunde für EOD
InpEODMinute	int	50	Minute für EOD
InpMagicNumber	int	123456	WICHTIG: Unique ID pro Chart!

Funktions-Referenz

OnInit()

Zweck: Initialisierung des EAs beim Start

Ablauf:

1. Setzt g_currentFilter auf InpStartDirection
2. Initialisiert stat_HighWaterMark mit aktueller Balance
3. Erstellt Stochastik-Handle (H1)

4. Erstellt RSI-Handle (D1) wenn InpEntryMode == MODE_RSI
5. Setzt Chart-Styling (schwarz/weiß)
6. Konfiguriert Trade-Objekt
7. Erstellt GUI (Buttons + Labels)
8. Aktualisiert Button-States

Return: INIT_SUCCEEDED oder INIT_FAILED

OnDeinit()

Zweck: Aufräumen beim EA-Stop

Ablauf:

1. Released Indikator-Handles
 2. Löscht alle GUI-Objekte (Buttons + Labels)
 3. Löscht Chart-Comment
-

OnChartEvent()

Zweck: Reagiert auf GUI-Button-Klicks

Parameter:

- id: Event-ID (CHARTEVENT_OBJECT_CLICK)
- sparam: Name des geklickten Objekts

Logik:

Wenn Button geklickt:

- └─ Setze g_currentFilter auf entsprechenden Wert
 - └─ UpdateButtonsState() - Färbe Buttons neu
 - └─ ValidatePositionsAgainstFilter() - Schließe ungültige Positionen
 - └─ ChartRedraw()
-

OnTick()

Zweck: Haupt-Logik, wird bei jedem Tick ausgeführt

Struktur: Siehe [Trading-Logik](#) oben.

CalculateADR_and_Range()

Zweck: Berechnet ADR und aktuelle Tagesrange

Algorithmus:

```
sum = 0;
for(i=1 bis InpADRRPeriod)
{
    sum += (High[i] - Low[i]); // D1
}
adrValue = sum / InpADRRPeriod;

dayRange = High[0] - Low[0]; // Aktuelle D1-Kerze
```

Fallback:

Wenn adrValue == 0 → adrValue = 100 * _Point

ValidatePositionsAgainstFilter()

Zweck: Schließt Positionen, die nicht zum aktuellen Filter passen

Logik:

Wenn Filter == BOTH oder NEUTRAL → Return (nichts tun)

Für jede Position:

- |– Wenn Symbol passt UND Magic passt:
- | |– Wenn Filter == LONG und Position == SELL → Schließen
- | |– Wenn Filter == SHORT und Position == BUY → Schließen

Use-Case: User hat 3 Short-Positionen offen und klickt auf "LONG" Button. → Alle Shorts werden sofort geschlossen.

GetDynamicLot()

Zweck: Berechnet dynamische Lot-Größe basierend auf Balance-Entwicklung

Parameter:

- baseLot: Basis-Lot-Größe (z.B. InpFirstLot oder InpGridLot)

Return: Berechnete Lot-Größe (normalisiert auf Broker-Steps)

Algorithmus:

Wenn InpUseMM == false → return baseLot

$\text{profitAbs} = \text{currentBalance} - \text{InpRefBalance}$

$\text{profitPct} = (\text{profitAbs} / \text{InpRefBalance}) * 100$

$\text{steps} = (\text{int})(\text{profitPct} / \text{InpBalanceStep_Pct})$

$\text{multiplier} = 1.0 + (\text{steps} * (\text{InpLotIncrease_Pct} / 100))$

$\text{calculatedLot} = \text{baseLot} * \text{multiplier}$

→ Normalisiere auf Broker-Step/Min/Max

Beispiel:

$\text{baseLot} = 0.02$

$\text{RefBalance} = 10.000\text{€}$

$\text{Balance} = 12.500\text{€ (+25\%)}$

$\text{BalanceStep} = 20\%$

$\text{LotIncrease} = 20\%$

$\text{steps} = 25 / 20 = 1$

$\text{multiplier} = 1 + (1 * 0.2) = 1.2$

→ Return: 0.024

CheckBasketExit()

Zweck: Prüft ob Basket-Exit-Bedingung erreicht ist

Parameter:

- type: POSITION_TYPE_BUY oder POSITION_TYPE_SELL
- count: Anzahl offener Positionen

Algorithmus:

1. Berechne gewichteten Durchschnittspreis aller Positionen

$\text{avgPrice} = \Sigma(\text{Preis} * \text{Volumen}) / \Sigma(\text{Volumen})$

2. Berechne dynamisches Target

$\text{targetPct} = \text{InpStartTarget_ADR} - ((\text{count} - 1) * \text{InpTargetDecay_ADR})$

$\text{targetDistPoints} = \text{adrValue} * (\text{targetPct} / 100)$

3. Prüfe Exit-Bedingung

Long: $\text{Bid} \geq \text{avgPrice} + \text{targetDistPoints}$

Short: $\text{Ask} \leq \text{avgPrice} - \text{targetDistPoints}$

4. Wenn erfüllt → CloseAllPositions()

CloseAllPositions()

Zweck: Schließt alle Positionen mit passendem Symbol und Magic

Ablauf:

Für alle Positionen (rückwärts iterieren!):

└─ Wenn $\text{Symbol} == _Symbol$

└─ UND $\text{Magic} == \text{InpMagicNumber}$

└─ → $\text{trade.PositionClose(ticket)}$

UpdateVisuals()

Zweck: Aktualisiert alle GUI-Labels

Parameter:

- positions: Anzahl offener Positionen

Aktualisiert:

1. **MaxDD Label:** "MaxDD zu HWM-Balance in %: XX.XX%"
2. **Positions Label:** "Anzahl an Positionen: X"
3. **Status Label:** "Status: [stat_EA_Action]"
4. **Setup Label:** "Config: Cap XXX | Grid XX% | Target XX% | Lot X.XX | Magic XXXXXX"

CreateButtons()

Zweck: Erstellt die 4 GUI-Buttons

Button-Positionen:

Y = 5px (ganz oben)

X = 20, 115, 210, 305 (je 90px breit + 5px gap)

[BOTH] [LONG] [SHORT] [NEUTRAL]

CreateSingleButton()

Zweck: Erstellt einen einzelnen Button

Parameter:

- name: Objekt-Name
- x, y: Position
- w, h: Breite/Höhe
- text: Button-Text

Properties:

- Type: OBJ_BUTTON
 - Color: clrBlack auf clrSilver
 - Font: 9pt
 - Nicht auswählbar, nicht verschiebbar
-

UpdateButtonsState()

Zweck: Färbt Buttons entsprechend des aktiven Filters

Farb-Schema:

Filter	Farbe
BOTH	clrSkyBlue
LONG	clrLimeGreen
SHORT	clrTomato
NEUTRAL	clrKhaki
Inaktiv	clrWhiteSmoke

CreateLabels()

Zweck: Erstellt die 4 Info-Labels

Label-Positionen:

X = 20px (links)

Y = 50, 75, 100, 125 (unter den Buttons)

MaxDD: Y = 50

Positionen: Y = 75

Status: Y = 100

Setup: Y = 125

Eigenschaften:

- Font: Arial Bold, 11pt
- Color: clrWhite
- Nicht auswählbar

CreateSingleLabel()

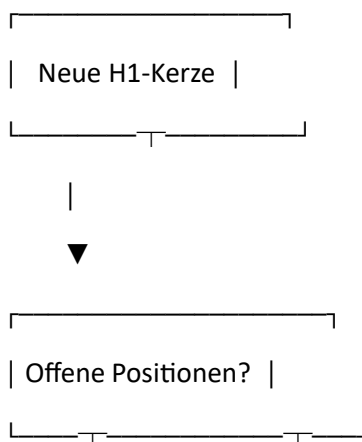
Zweck: Erstellt ein einzelnes Label

Parameter:

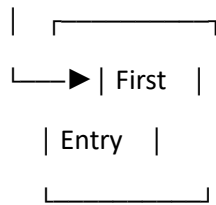
- name: Objekt-Name
- x, y: Position
- text: Initial-Text
- fSize: Font-Größe
- font: Font-Name
- c: Farbe

Ablaufdiagramme

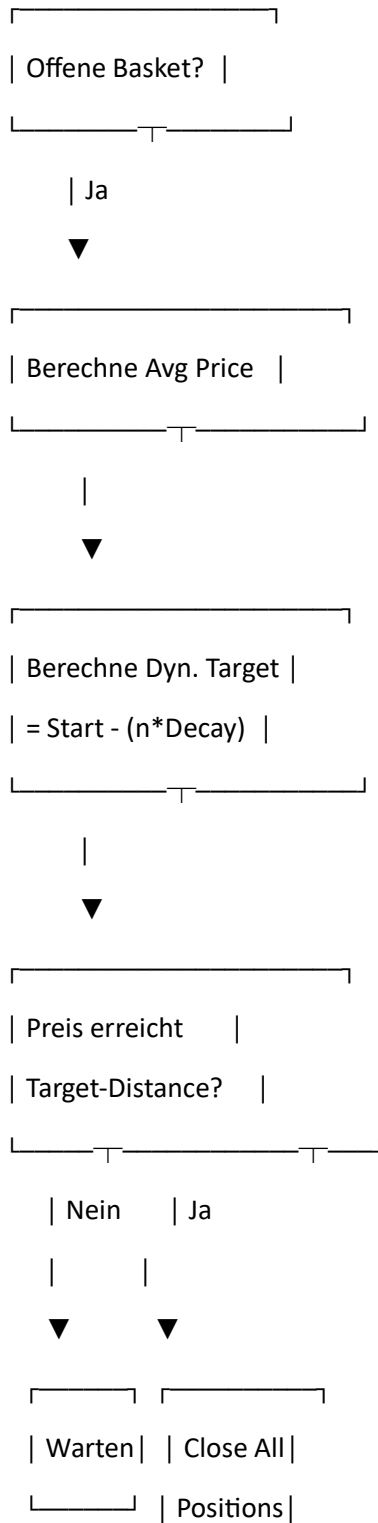
Entry-Ablauf



Nein	Ja → Grid-Check	
▼		
Filter OK?		
Nein	Ja	
▼	▼	
Entry-Modus	Preis bewegt	
ADR/RSI?	um Grid-Dist?	
		Ja
ADR	RSI	▼
▼	▼	Grid Buy
		or Sell
Range	RSI	
>X%?	Ext?	
Ja	Ja	
▼		
Stochastik		
Crossover?		
		Ja
▼		



Exit-Ablauf



GUI-Steuerung

Button-Funktionalität

BOTH Button (Blau)

- **Funktion:** Erlaubt Long UND Short Entries
- **Verhalten:**
 - Bestehende Positionen bleiben offen
 - Neue Entries in beide Richtungen möglich
 - Grid-Nachkäufe für beide Richtungen

LONG Button (Grün)

- **Funktion:** Nur Long Entries
- **Verhalten:**
 - **Alle Short-Positionen werden sofort geschlossen**
 - Nur Long-Entries möglich
 - Nur Long-Grid möglich
 - Short-Signale werden ignoriert

SHORT Button (Rot)

- **Funktion:** Nur Short Entries
- **Verhalten:**
 - **Alle Long-Positionen werden sofort geschlossen**
 - Nur Short-Entries möglich
 - Nur Short-Grid möglich
 - Long-Signale werden ignoriert

NEUTRAL Button (Khaki)

- **Funktion:** Pause-Modus
- **Verhalten:**
 - Bestehende Positionen bleiben offen
 - **Keine neuen Entries**
 - **Keine Grid-Nachkäufe**
 - Exit-Logik läuft weiter

- Nützlich um "nur zu managen ohne nachzukaufen"

Info-Labels

MaxDD Label

MaxDD zu HWM-Balance in %: XX.XX%

Zeigt maximalen Equity-Drawdown seit EA-Start.

Positionen Label

Anzahl an Positionen: X

Zeigt aktuelle Anzahl offener Positionen (nur eigene Magic).

Status Label

Status: [Dynamischer Text]

Zeigt aktuellen EA-Status:

- "Scanning for Entry..."
- "Basket Open (Long/Short). Next Grid: XX.XXXXX"
- "Wait Range (X < Y)"
- "Wait RSI (XX.X)"
- "ADR met. Wait Stoch..."
- "Entered Long/Short"
- "EOD Time. Closing/Waiting."

Setup Label

Config: Cap XXX | Grid XX% | Target XX% | Lot X.XX | Magic XXXXXX

Zeigt Hauptparameter:

- **Cap:** InpEntryADR_Pct (Entry-Schwelle)
- **Grid:** InpGridStep_ADR_Pct (Grid-Abstand)
- **Target:** InpStartTarget_ADR (Start-Target)
- **Lot:** InpFirstLot (Start-Lot)
- **Magic:** InpMagicNumber (für Multi-Chart)

Risiko-Management

Individueller Stop Loss

Jede Position erhält einen eigenen SL:

SL-Distanz = ADR * InpIndividualSL_ADR

Beispiel (Gold):

- ADR = 100 Points (10 USD)

- InpIndividualSL_ADR = 3.0

→ SL = 30 USD Abstand

Wichtig:

- SL wird **beim Entry gesetzt** (nicht dynamisch angepasst)
- Bei Grid-Nachkäufen erhält jede neue Position ihren eigenen SL
- SL schützt nur die einzelne Position, nicht das Basket

Basket-Exit

Das Basket wird als Ganzes geschlossen wenn:

Gewinn $\geq$ (Target% * ADR)

Target dynamisch:

- 1 Position: 10% ADR

- 2 Positionen: 5% ADR (10% - 1*5%)

- 3+ Positionen: 0% ADR → Breakeven-Exit

Money Management

Automatische Lot-Skalierung bei Profit:

Bei +20% Balance → +20% Lots

Bei +40% Balance → +40% Lots

usw.

Wichtig:

- Nur bei **positivem** Profit aktiv
- Niemals bei Verlust reduziert
- Optional (InpUseMM = false standardmäßig)

Max Positions

InpMaxPositions = 10 (default)

Limitiert Grid-Tiefe auf 10 Levels.

Risk-Calculation:

First Entry: 0.02 Lot

9x Grid: 0.01 Lot

= 0.11 Lot total

Bei 25 Points Grid-Abstand:

= $9 * 25 = 225$ Points maximale Distanz

= Worst Case bei 100 Point ADR: 2.25x ADR drawdown

End-of-Day Protection

Optional: Automatisches Schließen vor Tagesende

InpUseEODClose = true

InpEODHour = 23

InpEODMinute = 50

→ Um 23:50 werden alle Positionen geschlossen

→ Verhindert Overnight-Risiko

Empfohlene Einstellungen

Konservativ (niedriges Risiko)

InpFirstLot = 0.01

InpGridLot = 0.005

InpMaxPositions = 5

InpIndividualSL_ADR = 5.0

InpEntryADR_Pct = 200.0

InpGridStep_ADR_Pct = 30.0

InpStartTarget_ADR = 15.0

InpTargetDecay_ADR = 3.0

Standard (mittleres Risiko)

InpFirstLot = 0.02

InpGridLot = 0.01

InpMaxPositions = 10

InpIndividualSL_ADR = 3.0

InpEntryADR_Pct = 150.0

InpGridStep_ADR_Pct = 25.0

InpStartTarget_ADR = 10.0

InpTargetDecay_ADR = 5.0

Aggressiv (hohes Risiko)

InpFirstLot = 0.05

InpGridLot = 0.02

InpMaxPositions = 15

InpIndividualSL_ADR = 2.0

InpEntryADR_Pct = 100.0

InpGridStep_ADR_Pct = 20.0

InpStartTarget_ADR = 8.0

InpTargetDecay_ADR = 4.0

Wichtige Hinweise

Multi-Chart Setup

WICHTIG: Jeder Chart braucht eine **eigene Magic Number!**

Chart 1 (XAUUSD): InpMagicNumber = 100001

Chart 2 (EURUSD): InpMagicNumber = 100002

Chart 3 (GBPUSD): InpMagicNumber = 100003

Sonst würden alle Charts dieselben Positionen sehen und verwalten!

H1-Timeframe erforderlich

Der EA sollte auf **H1-Chart** laufen:

- Stochastik arbeitet auf H1
- Entry/Grid-Prüfung pro H1-Kerze
- Bei anderen Timeframes funktioniert Logik nicht korrekt

D1-Daten erforderlich

Für ADR/RSI-Berechnung werden **D1-Daten** benötigt:

- ADR: 14 Tage History
- RSI: Aktueller D1-RSI
- Sicherstellen dass genug History geladen ist

Filter-Wechsel während Basket

Wenn Filter geändert wird während Positionen offen:

Scenario: 3x Long offen, User klickt "SHORT"

→ Alle 3 Longs werden SOFORT geschlossen

→ Kann zu ungewollten Verlusten führen

Empfehlung: Filter nur wechseln wenn keine Positionen offen.

EOD-Close

Wenn aktiviert wird **hart** um die eingestellte Zeit geschlossen:

- Kein Warten auf Target
- Auch wenn Position stark im Minus
- Täglich um dieselbe Zeit

Empfehlung: Nur nutzen wenn Overnight-Risiko vermieden werden soll.

Performance-Metriken

Monitored Metrics

Der EA trackt automatisch:

1. **High Water Mark (HWM)**
 - Höchste erreichte Balance seit EA-Start
 - Referenz für DD-Berechnung
2. **Max Drawdown (Equity)**
 - In Geldeinheiten (z.B. 500 EUR)
 - In Prozent (z.B. 5%)
 - Bezogen auf HWM, nicht auf aktuelle Balance
3. **Position Count**
 - Aktuelle Anzahl offener Positionen
 - Nur Positionen mit eigener Magic

Nicht getrackt

- Win Rate
- Profit Factor
- Sharpe Ratio
- Max Consecutive Wins/Losses
- Average Trade Duration

Empfehlung: Für umfassendes Monitoring externes Tool nutzen (z.B. MyFXBook).

Bekannte Limitationen

1. **Kein Drawdown-Stop**
 - EA schließt nicht automatisch bei X% DD
 - Nur individuelle SLs pro Trade
2. **Keine Equity-Protection**
 - Kein Floating-Equity Stop
 - Basket läuft bis Target oder alle SLs
3. **Keine News-Filter**
 - EA tradet auch bei High-Impact News
 - Kann zu extremen Slippage führen
4. **Keine Spread-Filter**
 - Tradet auch bei hohen Spreads
 - Kann Profitabilität reduzieren
5. **Keine Recovery-Funktion**
 - Bei alle SLs getroffen: Kein Auto-Recovery
 - Neue Entries erst bei neuen Signalen
6. **H1-Bar Limitation**
 - Entry/Grid nur 1x pro H1-Kerze
 - Schnelle Moves innerhalb H1 werden verpasst

Upgrade-Möglichkeiten

Potenzielle Erweiterungen

- **Trailing Stop** für profitable Baskets
- **Breakeven-Move** nach X Points Profit
- **News-Filter** (Calendar Integration)
- **Spread-Filter** (max. Spread für Entry)
- **Time-Filter** (bestimmte Stunden deaktivieren)
- **Recovery-Mode** (spezielle Logik nach SL-Serie)
- **Multi-Timeframe** Bestätigung

- **Correlation-Filter** (nicht Long EUR/USD + Short GBP/USD gleichzeitig)
 - **Session-Filter** (nur London/NY Session)
 - **CSV-Export** für Performance-Daten
-

Changelog

Version 1.28

-  Magic Number Display im Setup-Label
-  Feature-Complete für Multi-Symbol Usage

Frühere Versionen

-  GUI-Buttons für Filter-Steuerung
 -  Dynamische Exit-Targets
 -  Money Management System
 -  EOD-Close Funktion
 -  Dual Entry-Mode (ADR/RSI)
 -  Individual SL pro Trade
 -  MaxDD Monitoring
-

Support & Wartung

Code-Wartung

- **Prozedurales Design:** Alle Funktionen sind global, keine Klassen
- **Einfache Erweiterung:** Neue Funktionen als globale Functions hinzufügen
- **GUI-Elemente:** Bei neuen Buttons/Labels: Namen in Globals definieren

Debugging

Wichtige Check-Points:

1. stat_EA_Action zeigt aktuellen Status
2. Setup-Label zeigt Konfiguration
3. Comment-Funktion verfügbar für zusätzliche Debug-Ausgabe
4. Expert-Log prüfen für Trade-Errors

Testing

Empfohlene Tests:

1. **Filter-Switches:** Alle 4 Button-Kombinationen testen

2. **Grid-Sequenz:** Bis InpMaxPositions Grid durchlaufen
3. **Exit-Logik:** Basket-Exit bei verschiedenen Levels testen
4. **MM-Funktion:** Mit verschiedenen Balance-Stufen testen
5. **EOD-Close:** Zeit-Trigger testen

Dokumentation erstellt: 2025

Autor: Algorithm Factory

Zweck: Technische Referenz für Entwickler & Trader

2 Prompt:

Du bist ein Experte für MQL5 Programmierung. Passe mir den Code für den Robot an.

Ich habe folgendes vor:

Der Robot soll auf mehreren Währungspaaren parallel laufen, hierzu installiere ich den Robot handisch auf mehreren Währungspaaren.

Der Robot soll so abgeändert werden dass er seine Traderichtung für sein aktuelles Symbol wo er installiert ist aus einer Konfig-Datei herausliest.

Er muss also feststellen auf welchem Währungspaar Chart er läuft und dann in der Datei nachschauen wie die Traderichtung ist. Wenn die Traderichtung Long ist dann muss er Long traden,

die Datei heißt last_known_signals.csv und befindet sich im Dateien-Verzeichnis von MetaTrader 5

Also in meinem Beispiel in F:\Forex\mt5\ActiveTradesReal\MQL5\Files\last_known_signals.csv

Hier mal ein Beispiel:

```
Währungspaar;Letztes_Signal
NZDUSD;BUY
AUDJPY;BUY
GBPJPY;BUY
USDJPY;NEUTRAL
EURCHF;SELL
EURUSD;BUY
EURAUD;SELL
USDCHF;SELL
GOLD;SELL
AUDUSD;BUY
GBPUSD;BUY
EURJPY;BUY
USDCAD;SELL
EURGBP;NEUTRAL
SILVER;SELL
GBPCHF;SELL
```

Du musst beachten dass die erste Zeile ausgeblendet werden muss wenn du die Datei auswertest.

Hier jetzt die Aufgabe.

- 1) Passe den Robot so an das er nachschaut ob eine last_known_signals.csv vorhanden ist. Wenn ja dann
- 2) Lese die datei ein. Schau nach auf welchen Währungspaar ich laufe und hole dir die passende Traderichtung heraus.
- 3) Handle nur die Trades in der entsprechenden richtung
- 4) Wenn Trades in der gegenrichtung offen sind schliesse die.
- 5) Schliesse aber nur bei gegensignal also wenn long trades offen sind und die datei sagt sell dann schliesse. Wenn da neutral drin steht dann mache noch nix.
- 6) Die datei last_known_signals.csv muss jede x-Minuten ausgewertet werden. Default wert ist 15 Minuten. Dieser wert muss im Robot konfigurierbar sein.
- 7) Also jede 15 minuten werte last_known_signals aus und merke dir die traderichtung in einer globalen oder internen variable.
- 8) Anhand dieser variable mache die passenden Trades nur in dieser richtung.
- 9) Gibt hinweise im chart aus wann das letzte mal diese last_known_signals.csv ausgewertet wurde und gib die traderichtung im chart für das aktuelle symbol aus.

Wenn last_known_signals.csv nicht vorhanden ist, dann gib eine warnung aus und dann darfst du die buttons im chart verwenden. Wenn die datei last_known_signals.csv vorhanden ist, darf man die buttons für die Traderichtung nicht verwenden können, graue die aus wenn das möglich ist.

3 Kursbeschreibung für den prompt

Prompt:

mache mir eine kurzbeschreibung des gesamten codes, so das du die verstehen kannst und ich dir diese beschreibung beim nächsten male als prompt wieder mitgeben kann.

CC ADR Stoch Grid Trading Robot - Code-Kurzbeschreibung (v1.30)

Zweck

Mean-Reversion Grid Trading System für MetaTrader 5 mit ADR/RSI-Entry, Stochastik-Timing, dynamischen Targets, CSV-basierter Signalsteuerung und MQL5 Economic Calendar News-Filter für Multi-Symbol-Betrieb.

Code-Struktur: PROZEDURAL (keine OOP-Klassen)

Haupt-Komponenten:

1. Input-Parameter (7 Gruppen, inkl. News-Filter)
2. Globale Variablen (Trading + Stats + CSV + News + GUI)
3. Event-Handler (OnInit, OnDeinit, OnTick, OnTimer, OnChartEvent)
4. Trading-Funktionen (Entry, Grid, Exit, MM)
5. CSV-Integration (Signalverarbeitung, Timer-basiert)
6. News-Filter (MQL5 Calendar API, Echtzeit-Prüfung)

7. GUI-System (4 Buttons + 7 Labels)
 8. Helper-Funktionen (ADR, Validierung, etc.)
-

Input-Parameter-Gruppen

1. CSV Signal Integration

- **InpUseCSVSignals (bool):** CSV-Mode aktivieren?
- **InpCSVCheckInterval (int):** Prüfintervall in Minuten (default: 15)
- **InpCSVFilename (string):** CSV-Dateiname (default: "last_known_signals.csv")

2. News Filter (MQL5 Calendar) - NEU in v1.30

- **InpUseNewsFilter (bool):** News-Filter aktivieren?
- **InpNewsImportance (ENUM_NEWS_IMPORTANCE):** Welche News filtern? (HIGH_ONLY oder MEDIUM_HIGH)
- **InpMinutesBeforeNews (int):** Minuten VOR News (default: 30)
- **InpMinutesAfterNews (int):** Minuten NACH News (default: 30)
- **InpNewsFilterBothCurrencies (bool):** Beide Währungen prüfen? (default: true)

3. Direction Filter

- **InpStartDirection:** Start-Filter bei EA-Init (wird von CSV überschrieben wenn aktiv)

4. Positions Management

- **InpFirstLot, InpGridLot:** Lot-Größen
- **InpMaxPositions:** Max Grid-Levels
- **InpIndividualSL_ADR:** SL pro Trade (Vielfaches von ADR)

5. Money Management

- **InpUseMM:** Balance-basierte Lot-Skalierung
- **InpRefBalance, InpBalanceStep_Pct, InpLotIncrease_Pct**

6. Entry Logic

- **InpEntryMode:** MODE_ADR (Range-basiert) oder MODE_RSI
- **ADR/RSI-Parameter:** Perioden, Thresholds

7. Exit + Time

- **InpStartTarget_ADR, InpTargetDecay_ADR:** Dynamische Targets
- **InpUseEODClose:** Optional End-of-Day Close
- **InpMagicNumber:** WICHTIG für Multi-Chart!

8. Stochastic (H1)

- InpStochK/D/Slowing, InpStochUpper/Lower: Für Entry-Timing

Wichtige Globale Variablen

Trading-Objekte

```
CTrade trade;           // Trade-Execution  
  
int handleStoch, handleRSI; // Indikator-Handles  
  
double adrValue, dayRange; // ADR-Berechnungen  
  
datetime lastProcessedH1Bar; // H1-Kerzen-Tracking
```

Statistik

```
double stat_HighWaterMark; // Höchste Balance  
  
double stat_MaxDD_Equity_Money/Percent; // Max Drawdown  
  
string stat_EA_Action; // Status-Text
```

Runtime Filter

```
ENUM_DIR_FILTER g_currentFilter; // Aktuelle Richtung (wird von CSV gesteuert)
```

CSV-Integration

```
bool g_CSVMode; // true wenn CSV-Datei gefunden  
  
ENUM_DIR_FILTER g_CSVDirection; // Richtung aus CSV  
  
datetime g_LastCSVCheck; // Letzter Check-Timestamp  
  
string g_CSVStatus; // Status-Text (z.B. "CSV erfolgreich gelesen")  
  
string g_CSVSignalText; // BUY/SELL/NEUTRAL/FILE NOT FOUND
```

News-Filter (NEU in v1.30)

```
bool g_NewsBlockActive; // true wenn News-Blockierung aktiv  
  
string g_NextNewsInfo; // Info-Text (z.B. "NFP [HIGH] in 15 Min")  
  
datetime g_NextNewsTime; // Zeit der nächsten relevanten News  
  
string g_NextNewsName; // Event-Name (z.B. "Non-Farm Payrolls")
```

GUI-Objekte

```
// Buttons: btnBothName, btnLongName, btnShortName, btnNeutralName  
  
// Labels: lblMaxDDName, lblPosName, lblStatusName, lblSetupName  
  
// CSV-Labels: lblCSVInfoName, lblCSVTimeName  
  
// News-Label (NEU): lblNewsInfoName
```

Event-Handler

OnInit()

1. Chart-Styling setzen (schwarz/weiß)
2. Indikatoren erstellen (Stoch, optional RSI)
3. GUI erstellen (Buttons + Labels)
4. Wenn InpUseCSVSignals = true:
 - CheckCSVFile() sofort aufrufen
 - Timer starten (EventSetTimer(InpCSVCheckInterval * 60))
5. Wenn InpUseNewsFilter = true: (NEU)
 - CheckUpcomingNews() initial aufrufen
 - Log-Ausgabe der News-Filter-Settings

OnDeinit()

1. Indikatoren freigeben
2. Timer stoppen (EventKillTimer())
3. Alle GUI-Objekte löschen (inkl. News-Label)

OnTick() - Haupt-Trading-Logik

Ablauf:

1. Monitoring: Balance, HWM, MaxDD berechnen
2. Preparations:
 - ValidatePositionsAgainstFilter() - Schließt ungültige Positionen
 - CalculateADR_and_Range()
 - CheckUpcomingNews() - Prüft News bei jedem Tick (NEU)
 - Zähle offene Positionen (nur eigene Magic)
3. Status Logic: Setze stat_EA_Action Text
4. Logic Execution:
 - Wenn Positionen > 0:
 - CheckBasketExit()
 - Grid-Nachkauf prüfen ABER nur wenn !g_NewsBlockActive (NEU)
 - Wenn EOD-Zeit: CloseAllPositions()
 - Wenn keine Positionen + neue H1-Kerze:
 - Wenn g_NewsBlockActive: Entry blockieren (NEU)

- Entry-Bedingung prüfen (ADR oder RSI)
- Stochastik-Crossover prüfen
- First Entry ausführen (wenn alle Bedingungen erfüllt)

5. Visuals: UpdateVisuals() aufrufen

OnTimer()

Wird alle X Minuten aufgerufen (InpCSVCheckInterval):

if(InpUseCSVSignals) CheckCSVFile();

OnChartEvent()

Button-Klicks verarbeiten:

- Wenn g_CSVMODE = true: Klicks blockieren, Warnung ausgeben
- Sonst: Filter ändern, ValidatePositionsAgainstFilter() aufrufen

Wichtige Funktionen

News-Filter (NEU in v1.30)

CheckUpcomingNews()

Zweck: Prüft bei jedem Tick ob relevante News anstehen oder kürzlich waren

Ablauf:

1. Wenn !InpUseNewsFilter → Return (deaktiviert)
2. Hole Trade-Server-Zeit (TimeTradeServer()) - WICHTIG!
3. Definiere Zeitfenster:
 - dateFrom = currentTime - (InpMinutesAfterNews * 60)
 - dateTo = currentTime + (InpMinutesBeforeNews * 60)
4. Extrahiere Währungen:
 - currencyBase = SymbolInfoString(_Symbol, SYMBOL_CURRENCY_BASE) (z.B. "EUR")
 - currencyQuote = SymbolInfoString(_Symbol, SYMBOL_CURRENCY_PROFIT) (z.B. "USD")
5. Setze Mindest-Wichtigkeit:
 - NEWS_HIGH_ONLY → CALENDAR_IMPORTANCE_HIGH
 - NEWS_MEDIUM_HIGH → CALENDAR_IMPORTANCE_MODERATE
6. Rufe CalendarValueHistory() für Base-Währung auf
7. Optional: Rufe CalendarValueHistory() für Quote-Währung auf (wenn InpNewsFilterBothCurrencies)

8. Loop durch Events:

- Hole Event-Details mit `CalendarEventById()`
- Prüfe `event.time_mode` (nur DATETIME oder DATE)
- Prüfe `event.importance >= minImportance`
- Speichere nächstes/nahestes Event

9. Wenn News gefunden:

- `g_NewsBlockActive = true`
- `g_NextNewsTime = nearestNewsTime`
- `g_NextNewsName = event.name`
- Berechne Minuten bis/seit News
- Setze `g_NextNewsInfo` (z.B. "NFP [HIGH] in 15 Min")

10. Sonst:

- `g_NewsBlockActive = false`
- `g_NextNewsInfo = "Klar - Keine News"`

Wichtige MQL5 API-Funktionen:

`CalendarValueHistory(values, dateFrom, dateTo, NULL, currency)` // Hole Events

`CalendarEventById(event_id, event)` // Hole Event-Details

Event-Wichtigkeit (Enum):

`CALENDAR_IMPORTANCE_NONE = 0`

`CALENDAR_IMPORTANCE_LOW = 1`

`CALENDAR_IMPORTANCE_MODERATE = 2` // WICHTIG: MODERATE nicht MEDIUM!

`CALENDAR_IMPORTANCE_HIGH = 3`

Trading-Blockierung:

- In `OnTick` wird `g_NewsBlockActive` geprüft
- Wenn true: Kein First Entry, kein Grid-Nachkauf
- Offene Positionen bleiben bestehen (werden NICHT geschlossen)

CSV-Integration

`CheckCSVFile()`

Zweck: CSV-Datei lesen und Signal für aktuelles Symbol holen Ablauf:

1. Öffne CSV aus `MQL5/Files/` Verzeichnis
2. Wenn nicht gefunden: `g_CSVMode = false`, Warnung, RETURN

3. Erste Zeile überspringen (Header)
4. Alle Zeilen durchgehen, nach _Symbol suchen
5. Signal parsen: "BUY" → FILTER_LONG, "SELL" → FILTER_SHORT, "NEUTRAL" → FILTER_NEUTRAL
6. Wenn gefunden: g_CSVMode = true, g_currentFilter setzen
7. Wenn Richtungswechsel: CloseOppositePositions() aufrufen
8. UpdateButtonsState() aufrufen

CloseOppositePositions(oldDir, newDir)

Zweck: Gegenpositionen schließen bei Signalwechsel Regeln:

- NEUTRAL als Ziel: NICHTS schließen
- Von NEUTRAL: NICHTS schließen (keine Positionen)
- BUY → SELL: Alle Long-Positionen schließen
- SELL → BUY: Alle Short-Positionen schließen

Trading-Logik

CalculateADR_and_Range()

Berechnet:

- adrValue = Durchschnitt der letzten X Tagesranges (D1)
- dayRange = Aktuelle Tagesrange (High[0] - Low[0])

ValidatePositionsAgainstFilter()

Zweck: Schließt Positionen die nicht zum aktuellen Filter passen

- Bei FILTER_LONG: Shorts schließen
- Bei FILTER_SHORT: Longs schließen
- Bei FILTER_BOTH/NEUTRAL: Nichts tun

GetDynamicLot(baseLot)

MM-Funktion: Skaliert Lots basierend auf Balance-Entwicklung

Wenn Balance > RefBalance:

$\text{profitPct} = ((\text{Balance} - \text{Ref}) / \text{Ref}) * 100$

$\text{steps} = \text{profitPct} / \text{BalanceStep}$

$\text{multiplier} = 1 + (\text{steps} * \text{LotIncrease} / 100)$

return baseLot * multiplier (normalisiert)

CheckBasketExit(type, count)

Basket-Exit-Logik:

1. Berechne gewichteten Durchschnittspreis
2. Berechne dynamisches Target: $\text{StartTarget} - ((\text{count}-1) * \text{Decay})$
3. Wenn Preis Target erreicht: `CloseAllPositions()`

`CloseAllPositions()`

Schließt alle Positionen mit passendem Symbol + Magic

GUI-Funktionen

`UpdateButtonsState()`

Wenn `g_CSVMode = true`:

- Alle Buttons ausgrauen (`clrDarkGray`)
- Button-Text ändern zu "XXX (CSV)"
- Aktiver Filter trotzdem sichtbar (grau statt farbig)

Wenn `g_CSVMode = false`:

- Normale Farben: Blau/Grün/Rot/Khaki
- Normale Texte: "BOTH", "LONG", etc.

`UpdateVisuals(positions)` (ERWEITERT in v1.30)

Aktualisiert 7 Labels:

1. MaxDD
2. Position Count
3. Status
4. Setup/Config
5. CSV Info: Mode/Signal/Status mit Farbe (Grün=OK, Orange=Inaktiv)
6. CSV Time: Letzte Prüfung + Interval
7. News Info (NEU): News-Filter Status mit Farbe (Rot=blockiert, Grün=klar, Grau=aus)

`CreateButtons()`, `CreateLabels()`

Erstellen GUI-Elemente beim Init

 Trading-Logik-Flow (vereinfacht)

Entry

Neue H1-Kerze + Keine Positionen

→ Filter erlaubt Entry?

→ News-Filter aktiv? (NEU) → STOP

- Entry-Bedingung erfüllt? (ADR > X% ODER RSI Extrem)
- Stochastik-Crossover? (K kreuzt über/unter Threshold)
- First Entry (Long/Short) mit dynStartLot + individueller SL

Grid

Basket offen + Neue H1-Kerze

- Filter erlaubt Grid?
- News-Filter aktiv? (NEU) → STOP
- Preis bewegt um Grid-Distanz gegen Position?
- Grid-Nachkauf (in Basket-Richtung) mit dynGridLot + individueller SL

Exit

Bei jedem Tick:

- Berechne Avg-Price des Baskets
- Berechne dynamisches Target (abhängig von Grid-Level)
- Wenn Preis Target erreicht: CloseAllPositions()

ODER:

- Individueller SL getroffen (pro Position einzeln)

ODER (optional):

- EOD-Zeit erreicht: CloseAllPositions()

CSV-Workflow

Setup

1. EA auf mehreren Charts installieren (EURUSD, GOLD, GBPUSD, etc.)
2. Jeder Chart braucht eigene Magic Number!
3. CSV-Datei erstellen in: MQL5\Files\last_known_signals.csv
4. Format:
5. Währungspaar;Letztes_SignalEURUSD;BUYGOLD;SELLGBPUSD;NEUTRAL

Runtime

EA Start

- CheckCSVFile() sofort
- Timer startet (alle 15 Min)

Alle 15 Min:

- OnTimer() ausgelöst
- CheckCSVFile()
- Symbol in CSV suchen
- Signal holen
- g_currentFilter setzen
- Bei Wechsel: CloseOppositePositions()
- GUI aktualisieren

OnTick:

- Nutzt g_currentFilter für Entry-Entscheidungen
- Nutzt g_currentFilter für Grid-Entscheidungen

Fehlerbehandlung

CSV nicht gefunden:

- g_CSVMode = false
- Warnung in Log + Chart
- Buttons bleiben nutzbar (Fallback auf manuelle Steuerung)

Symbol nicht in CSV:

- g_CSVMode = true (Datei existiert)
- g_CSVDirection = FILTER_NEUTRAL
- Warnung in Chart
- Keine Trades

News-Filter-Workflow (NEU in v1.30)

Setup

InpUseNewsFilter = true

InpNewsImportance = NEWS_HIGH_ONLY (oder NEWS_MEDIUM_HIGH)

InpMinutesBeforeNews = 30

InpMinutesAfterNews = 30

InpNewsFilterBothCurrencies = true

Runtime

OnTick() bei jedem Tick

↓

CheckUpcomingNews()

↓

Zeitfenster: [jetzt-30Min bis jetzt+30Min]

↓

CalendarValueHistory() für EUR (Base)

CalendarValueHistory() für USD (Quote) - optional

↓

Events filtern:

- Nur zeitbasierte Events (DATETIME/DATE)

- Nur Wichtigkeit >= minImportance

↓

News gefunden?

└─ JA:

| └─ g_NewsBlockActive = true

| └─ g_NextNewsInfo = "NFP [HIGH] in 15 Min"

| └─ stat_EA_Action = "Entry blockiert: ..."

| └─ Kein First Entry, kein Grid

|

└─ NEIN:

└─ g_NewsBlockActive = false

└─ g_NextNewsInfo = "Klar - Keine News"

└─ Normal Trading

Event-Filterung

Wichtigkeit:

NEWS_HIGH_ONLY:

- minImportance = CALENDAR_IMPORTANCE_HIGH
- Filtert nur wichtigste News (NFP, Zinsentscheidungen, etc.)

NEWS_MEDIUM_HIGH:

- minImportance = CALENDAR_IMPORTANCE_MODERATE
- Filtert mittlere + hohe News (mehr Events)

Zeit-Modus:

CALENDAR_TIMEMODE_DATETIME → Akzeptiert (konkrete Uhrzeit)

CALENDAR_TIMEMODE_DATE → Akzeptiert (Datum mit Zeit)

Andere → Ignoriert (ganztägig)

Währungs-Check:

InpNewsFilterBothCurrencies = true:

- Prüft Base-Währung (z.B. EUR bei EURUSD)
- Prüft Quote-Währung (z.B. USD bei EURUSD)
- Blockiert wenn EINE davon News hat

InpNewsFilterBothCurrencies = false:

- Prüft nur Base-Währung

Chart-Anzeige

News-Filter: BLOCKIERT | NFP [HIGH] in 15 Min ← ROT (g_NewsBlockActive = true)

News-Filter: Klar - Keine News ← GRÜN (g_NewsBlockActive = false)

News-Filter: DEAKTIVIERT ← GRAU (InpUseNewsFilter = false)

Status-Text-Beispiele

"Entry blockiert: NFP [HIGH] in 15 Min"

"Grid blockiert: ECB Speech [HIGH] vor 5 Min"

"Scanning for Entry..." (wenn klar)

Besonderheiten

Multi-Chart-Fähigkeit

WICHTIG: Jeder Chart braucht unique Magic Number!

EURUSD: InpMagicNumber = 100001

GOLD: InpMagicNumber = 100002

GBPUSD: InpMagicNumber = 100003

Timer-System

- **EventSetTimer()** in OnInit (Sekunden: Interval * 60)
- **EventKillTimer()** in OnDeinit
- **OnTimer()** wird automatisch aufgerufen

CSV-Parsing

- **Trennzeichen: ; (Semikolon)**
- **Erste Zeile (Header) wird übersprungen**
- **StringSplit()** zum Parsen
- **StringTrimLeft/Right()** für saubere Werte

News-Filter (NEU)

- **Nutzt TimeTradeServer() NICHT TimeCurrent() (wichtig!)**
- **Wird bei jedem Tick aufgerufen (sehr performant)**
- **Blockiert nur neue Trades, schließt keine offenen Positionen**
- **Nutzt MQL5 Calendar API (keine DLL, keine externen APIs)**

GUI-Blockierung

Wenn CSV-Mode aktiv:

- **Button-Klicks werden in OnChartEvent() blockiert**
- **Warnung im Log**
- **Button-State wird auf false zurückgesetzt**

Chart-Layout (v1.30)

Y=5: [BOTH] [LONG] [SHORT] [NEUTRAL] ← Buttons (ausgegraut wenn CSV-Mode)

Y=50: MaxDD zu HWM-Balance in %: X.XX%

Y=75: Anzahl an Positionen: X

Y=100: Status: Entry blockiert: NFP [HIGH] in 15 Min ← Zeigt auch News-Blockierung

Y=125: Config: Cap X | Grid X% | Target X% | Lot X.XX | Magic XXXXXX

Y=150: CSV-Mode: [AKTIV/INAKTIV] | Signal: [BUY/SELL/NEUTRAL] | Status: [...]

Y=175: Letzte CSV-Prüfung: [DateTime] (Interval: X Min)

Y=200: News-Filter: [BLOCKIERT/Klar/DEAKTIVIERT] | [Event-Info] ← NEU

Label-Farben:

- CSV Info: Grün=aktiv, Orange=inaktiv, Grau=deaktiviert
 - News Info: Rot=blockiert, Grün=klar, Grau=deaktiviert
-

Wichtige Code-Stellen für Anpassungen

Neue Features hinzufügen

- Input-Parameter: Oben im input group Block
- Globale Variablen: Nach `//--- GLOBALS ---`
- Neue Funktionen: Am Ende vor `//+-----+`
- GUI erweitern: In `CreateLabels()` oder `CreateButtons()`

CSV-Logik ändern

- Parsing: In `CheckCSVFile()`
- Signal-Mapping: In `CheckCSVFile()` (BUY/SELL/NEUTRAL → Filter)
- Gegenpositions-Regeln: In `CloseOppositePositions()`

News-Filter-Logik ändern (NEU)

- Zeitfenster: In `CheckUpcomingNews()` (dateFrom/dateTo Berechnung)
- Wichtigkeits-Filter: In `CheckUpcomingNews()` (minImportance)
- Währungs-Check: In `CheckUpcomingNews()` (Base/Quote Logik)
- Blockierungs-Check: In `OnTick()` (if `g_NewsBlockActive`)

Entry-Logik ändern

- Bedingungen: In `OnTick()` nach "LOGIC EXECUTION"
 - ADR-Berechnung: In `CalculateADR_and_Range()`
 - Stochastik-Check: In `OnTick()` Grid/Entry-Sektion
-

Version Info

- v1.28: Original mit GUI-Buttons
- v1.29: + CSV-Integration, Timer-System, Button-Blockierung, erweiterte GUI
- v1.30: + MQL5 Economic Calendar News-Filter, News-Blockierung für Entry/Grid

Dateiname

CC_ADR_Stoch_Grid_with_News_Filter_v130.mq5

Änderungen in v1.30 (Summary)

Neue Enums

```
enum ENUM_NEWS_IMPORTANCE
{
    NEWS_HIGH_ONLY = 0,    // Nur High Impact
    NEWS_MEDIUM_HIGH = 1   // Medium + High Impact
};
```

Neue Input-Parameter (5 Stück)

1. InpUseNewsFilter
2. InpNewsImportance
3. InpMinutesBeforeNews
4. InpMinutesAfterNews
5. InpNewsFilterBothCurrencies

Neue Globale Variablen (4 Stück)

1. g_NewsBlockActive
2. g_NextNewsInfo
3. g_NextNewsTime
4. g_NextNewsName

Neue Funktionen (1 Stück)

1. CheckUpcomingNews() - Hauptfunktion für News-Filter

Neue GUI-Elemente (1 Label)

1. lblNewsInfoName (Y=200) - Zeigt News-Filter Status

Modifizierte Funktionen

1. OnInit() - Ruft CheckUpcomingNews() initial auf
2. OnDeinit() - Löscht News-Label
3. OnTick() - Ruft CheckUpcomingNews() bei jedem Tick auf, prüft g_NewsBlockActive vor Entry/Grid
4. UpdateVisuals() - Aktualisiert News-Label mit Farbe
5. CreateLabels() - Erstellt News-Label

MTL5 Calendar API Nutzung

CalendarValueHistory(values, dateFrom, dateTo, NULL, currency)

CalendarEventById(event_id, event)

TimeTradeServer() // Wichtig statt TimeCurrent()!

Wichtigkeits-Konstanten (CORRECT!)

CALENDAR_IMPORTANCE_NONE = 0

CALENDAR_IMPORTANCE_LOW = 1

CALENDAR_IMPORTANCE_MODERATE = 2 // NICHT MEDIUM!

CALENDAR_IMPORTANCE_HIGH = 3

Empfohlene Settings (v1.30)

Standard (empfohlen):

CSV:

InpUseCSVSignals = true

InpCSVCheckInterval = 15

News:

InpUseNewsFilter = true

InpNewsImportance = NEWS_HIGH_ONLY

InpMinutesBeforeNews = 30

InpMinutesAfterNews = 30

InpNewsFilterBothCurrencies = true

Trading:

InpFirstLot = 0.02

InpGridLot = 0.01

InpMaxPositions = 10

Konservativ (weniger News-Blockierung):

News:

InpNewsImportance = NEWS_HIGH_ONLY

InpMinutesBeforeNews = 60

InpMinutesAfterNews = 60

Aggressiv (mehr News-Blockierung):

News:

InpNewsImportance = NEWS_MEDIUM_HIGH

InpMinutesBeforeNews = 15

InpMinutesAfterNews = 15

4) Einbaue fonic news

Prompt:

jetzt braue mich noch was ein das wenn news anstehen der robot 30 minuten vorher und 30 minuten nachher nicht zusätzliche trades aufmacht. Mache mir erst mal vorschläge was man machen könnte. gut wäre es schon was fertiges einzubinden, da gibt es ja kostenlose dlls, schaue mal nach was es gibt und was man einfach einbinden könnte.

5 Kurzbeschreibung für Menschen

Prompt:

Mache mir eine beschreibung mit einfachen worten so als kurzbeschreibung für einen menschen was der robot macht.

ADR Grid Trading Robot - Einfache Erklärung

Was macht dieser Robot?

Dieser Robot ist ein **automatischer Handelsassistent** für den Forex-Markt (Devisenhandel). Er arbeitet nach einer speziellen Strategie namens "Mean Reversion" - das bedeutet: Er geht davon aus, dass Preise, die zu weit in eine Richtung gelaufen sind, meistens wieder zurückkommen.

Die Grundidee

Stell dir vor:

- Ein Währungspaar (z.B. Euro/Dollar) bewegt sich normalerweise jeden Tag um eine bestimmte Strecke - das ist die "durchschnittliche tägliche Bewegung" (ADR).
- Wenn der Preis heute schon **viel weiter** als normal gelaufen ist, wird er wahrscheinlich wieder zurückkommen.
- **Genau hier** steigt der Robot ein!

Beispiel:

- Euro/Dollar bewegt sich normalerweise 100 Punkte pro Tag
- Heute ist er schon 150 Punkte gestiegen → **Überdehnung!**

- Der Robot wartet auf ein Signal und verkauft dann Euro (weil er erwartet, dass der Preis zurückkommt)
-

Wie funktioniert das Trading?

1. Einstieg (Entry)

Der Robot wartet auf **drei Bedingungen gleichzeitig**:

A) Überdehnte Bewegung (ADR)

- Der Markt muss heute schon weiter gelaufen sein als normal
- Standard: 150% der durchschnittlichen Tagesbewegung
- Das zeigt: "Der Markt ist überhitzt"

B) Bestätigung (Stochastik)

- Ein technischer Indikator zeigt, dass der Markt "überkauft" oder "überverkauft" ist
- Das ist wie ein zweiter Check: "Ja, der Preis ist wirklich zu weit"

C) Richtungs-Erlaubnis

- Entweder aus einer Datei (automatisch)
- Oder über Buttons im Chart (manuell)
- So kannst du steuern: "Heute nur kaufen" oder "Heute nur verkaufen"

Wenn alle drei Bedingungen erfüllt sind: → Der Robot eröffnet den **ersten Trade** (z.B. 0.02 Lot)

2. Nachkäufe (Grid System)

Was ist, wenn der Preis weiter gegen dich läuft?

Der Robot kauft **automatisch nach** - das nennt sich "Grid Trading":

Beispiel:

1. Erste Position: Verkauf bei 1.1000
2. Preis steigt auf 1.1025 → Robot verkauft nochmal (2. Position)
3. Preis steigt auf 1.1050 → Robot verkauft nochmal (3. Position)
4. ... und so weiter (max. 10 Positionen)

Warum macht er das?

- Durch Nachkäufe verbessert sich der Durchschnittspreis
- Wenn der Preis dann zurückkommt, macht er schneller Gewinn
- Das ist wie "Durchschnittskosten-Effekt" beim Aktien-Sparen

Wichtig:

- Jede Position hat einen eigenen Stop-Loss (Absicherung)
 - Maximum ist konfigurierbar (Standard: 10 Positionen)
-

3. Ausstieg (Exit)

Wann schließt der Robot alle Positionen?

Der Robot berechnet einen **intelligenten Ausstieg**:

Dynamisches Gewinnziel:

- Bei 1 Position: Ziel ist 10% der ADR
- Bei 2 Positionen: Ziel ist 5% der ADR
- Bei 3+ Positionen: Ziel ist 0% der ADR (Breakeven)

Beispiel:

- Du hast 3 Positionen offen
- Durchschnittspreis: 1.1025
- Gewinnziel: Breakeven (weil 3 Positionen)
- Sobald Preis zurück auf 1.1025 → **Alle Positionen werden geschlossen**

Warum wird das Ziel kleiner?

- Je mehr Positionen, desto riskanter
 - Lieber kleiner Gewinn als großer Verlust
 - Bei vielen Positionen reicht "Breakeven" schon
-

Absicherungen & Schutz

1. Stop-Loss pro Trade

- **Jede einzelne Position** hat einen eigenen Stop-Loss
- Standard: 3x ADR Abstand
- Beispiel: Wenn ADR = 100 Punkte, dann SL bei 300 Punkten
- Das begrenzt den maximalen Verlust pro Position

2. News-Filter

Der Robot hat einen **eingebauten News-Schutz**:

Was macht er?

- Schaut automatisch im Wirtschaftskalender nach wichtigen News
- Standard: 30 Minuten vor und 30 Minuten nach wichtigen News

- In dieser Zeit: **KEINE neuen Trades!**

Warum ist das wichtig?

- Bei wichtigen News (z.B. Zinsentscheidung, NFP) kann der Markt verrückt spielen
- Preise bewegen sich zu schnell und unberechenbar
- Der Robot wartet ab, bis die News vorbei ist

Beispiel:

- 14:30 Uhr kommt NFP (wichtige US-Arbeitsmarkt-Daten)
- Ab 14:00 Uhr: Robot macht keine neuen Trades
- Bis 15:00 Uhr: Robot wartet ab
- Ab 15:00 Uhr: Normal weiter

Was du einstellen kannst:

- Nur hochwertige News (NFP, Zinsen, etc.)
- Oder auch mittelwichtige News
- Zeitfenster: 15, 30, 60 Minuten (wie du willst)

3. CSV-Datei Steuerung

Du kannst eine einfache Textdatei nutzen um **alle Charts gleichzeitig** zu steuern:

Datei-Format:

Währungspaar;Richtung

EURUSD;BUY

GOLD;SELL

GBPUSD;NEUTRAL

Was passiert?

- Der Robot liest diese Datei alle 15 Minuten
- Sieht nach was für sein Währungspaar drinsteht
- EURUSD → Nur Long-Trades
- GOLD → Nur Short-Trades
- GBPUSD → Keine Trades

Vorteil:

- Du steuerst 10 Charts mit einer Datei
- Perfekt wenn du ein externes Signal-System hast
- Keine manuelle Bedienung pro Chart nötig

4. End-of-Day Close (Optional)

- Alle Positionen werden automatisch geschlossen vor Handelsschluss
 - Standard: 23:50 Uhr
 - Vermeidet Overnight-Risiko
-

Money Management

Balance-Skalierung (Optional)

Der Robot kann automatisch die Lot-Größen anpassen:

Beispiel:

- Startkapital: 10.000 EUR
- Start-Lot: 0.02
- Bei 12.000 EUR (+20%) → Lot wird 0.024
- Bei 14.000 EUR (+40%) → Lot wird 0.028
- usw.

Vorteil:

- Bei Gewinn werden Lots größer (mehr Profit)
 - Bei Verlust bleiben Lots gleich (weniger Risiko)
 - Automatisches Risiko-Management
-

Bedienung

Buttons im Chart

Der Robot hat 4 Buttons oben links im Chart:

[BOTH] - Grün wenn aktiv

- Erlaubt Long UND Short Trades
- Normale Betriebsart

[LONG] - Grün wenn aktiv

- Nur Long-Trades (Kaufen)
- Alle offenen Short-Positionen werden geschlossen

[SHORT] - Rot wenn aktiv

- Nur Short-Trades (Verkaufen)
- Alle offenen Long-Positionen werden geschlossen

[NEUTRAL] - Gelb wenn aktiv

- KEINE neuen Trades
- Offene Positionen laufen weiter
- Pause-Modus

Wichtig:

- Wenn CSV-Datei aktiv ist, sind Buttons ausgegraut
 - Dann steuert die Datei, nicht die Buttons
-

Anzeige im Chart

Der Robot zeigt dir **live** folgende Infos:

Zeile 1: Drawdown

- Maximaler Verlust seit Start
- In Prozent zur höchsten Balance

Zeile 2: Offene Positionen

- Wie viele Trades gerade laufen
- Zeigt Grid-Tiefe

Zeile 3: Status

- Was macht der Robot gerade?
- "Scanning for Entry" = Sucht nach Signal
- "Entry blockiert: NFP in 15 Min" = Wartet wegen News
- "Managing Basket" = Verwaltet offene Positionen

Zeile 4: Einstellungen

- Zeigt deine wichtigsten Parameter
- Lot-Größe, Magic Number, etc.

Zeile 5: CSV-Status

- Ist CSV-Datei aktiv?
- Welches Signal steht drin?
- Wann wurde zuletzt geprüft?

Zeile 6: CSV-Zeit

- Wann war die letzte Prüfung?
- Alle wieviel Minuten wird geprüft?

Zeile 7: News-Filter 🔥

- **GRÜN:** "Klar - Keine News" → Trading erlaubt
 - **ROT:** "BLOCKIERT | NFP in 15 Min" → Kein Trading
 - **GRAU:** "DEAKTIVIERT" → News-Filter aus
-

⚙️ Wichtige Einstellungen

Für Anfänger (Konservativ):

Start-Lot: 0.01

Grid-Lot: 0.005

Max Positionen: 5

ADR-Einstieg: 200% (nur wenn sehr überdehnt)

News-Filter: Nur hochwichtige News

Zeitfenster News: 60 Min vorher/nachher

Standard (Empfohlen):

Start-Lot: 0.02

Grid-Lot: 0.01

Max Positionen: 10

ADR-Einstieg: 150%

News-Filter: Nur hochwichtige News

Zeitfenster News: 30 Min vorher/nachher

Aggressiv (Erfahrene Trader):

Start-Lot: 0.05

Grid-Lot: 0.02

Max Positionen: 15

ADR-Einstieg: 100% (häufigere Einstiege)

News-Filter: Mittlere + hohe News

Zeitfenster News: 15 Min vorher/nachher

⚠️ Risiken & Wichtiges

Das musst du wissen:

1. Grid-Trading ist riskant

- Bei starken Trends können viele Positionen aufgehen
- Drawdown kann groß werden
- Nur mit Kapital handeln das du verlieren kannst

2. Magic Number ist wichtig

- Wenn du mehrere Charts nutzt: **Jeder Chart braucht eigene Magic Number!**
- Sonst verwechselt der Robot die Trades
- EURUSD: 100001, GOLD: 100002, etc.

3. News-Filter ist dein Freund

- Lass ihn AN!
- News können die Strategie zerstören
- Lieber ein Trade verpasst als großer Verlust

4. Backtesting

- Teste erst im Demo-Account
- Mindestens 1-2 Monate
- Schau dir den maximalen Drawdown an
- Nur wenn du damit leben kannst → Live

5. Nicht auf allen Paaren gleichzeitig

- Nicht auf korrelierten Paaren (EURUSD + GBPUSD)
- Diversifiziere: Gold, Forex, Indizes
- Sonst hast du 10x das gleiche Risiko

Praktische Tipps

Für besten Erfolg:

1. Wähle die richtigen Paare

- Gut: EURUSD, GBPUSD, GOLD, USDJPY
- Vermeide: Exotische Paare mit großem Spread
- Achte auf ausreichend Bewegung (hohe ADR)

2. Zeitfenster beachten

- Am besten während London/New York Session
- Asien-Session oft zu ruhig
- Nutze optional EOD-Close

3. Überwache am Anfang

- Erste Woche täglich checken
- Schau dir alle Trades an
- Lerne wie der Robot reagiert

4. Passe Parameter an

- Nicht die Standardwerte blind übernehmen
- Teste verschiedene Grid-Abstände
- Finde was zu deinem Risiko passt

5. Dokumentiere

- Screenshot von Einstellungen machen
- Ergebnisse notieren
- Was funktioniert? Was nicht?

Zusammenfassung in 3 Sätzen

Der Robot handelt "Mean Reversion": Er wartet bis der Markt überdehnt ist, steigt dann gegen die Bewegung ein und kauft bei Gegenbewegung nach (Grid). Er schließt alle Positionen wenn der Preis zurückkommt und ein Gewinnziel erreicht ist. News werden automatisch vermieden und du kannst alles über eine Datei oder Buttons steuern.

Häufige Fragen

F: Brauche ich VPS (Server)? A: Empfohlen! Der Robot muss 24/7 laufen um Signale nicht zu verpassen.

F: Wie viel Startkapital? A: Minimum 1000 EUR/USD pro Paar. Besser 2000-5000 EUR für Puffer.

F: Funktioniert auf allen Timeframes? A: Nein! Der Robot läuft auf H1 (1-Stunden-Chart). Nicht ändern!

F: Kann ich mehrere Paare gleichzeitig handeln? A: Ja! Aber jedes Paar braucht eigene Magic Number.

F: Was wenn CSV-Datei fehlt? A: Robot zeigt Warnung und nutzt Button-Steuerung als Fallback.

F: Muss ich bei jedem Trade dabei sein? A: Nein! Das ist ein vollautomatischer Robot. Aber überwache ihn anfangs.

F: Kann ich den Robot stoppen? A: Ja, einfach vom Chart entfernen. Offene Positionen bleiben.

F: Werden Positionen bei News geschlossen? A: Nein! Nur neue Trades werden blockiert. Offene laufen weiter.

Für wen ist dieser Robot?

Geeignet für:

✅ Trader die Mean-Reversion-Strategien mögen ✅ Leute die 24/7 automatisch handeln wollen ✅
Trader mit Risiko-Bewusstsein (Grid = riskant!) ✅ Menschen die Parameter verstehen und anpassen
können ✅ Trader die News-Events respektieren

Nicht geeignet für:

❌ Absolute Anfänger ohne Trading-Wissen ❌ Leute die "schnell reich werden" wollen ❌ Trader
mit zu kleinem Kapital (<1000 EUR) ❌ Menschen die nicht bereit sind zu testen ❌ Trader die kein
Risiko eingehen wollen

Wichtigster Rat: Teste erst ausgiebig im Demo! Verstehe was der Robot macht. Akzeptiere dass es
Drawdowns gibt. Nur dann bist du bereit für Live-Trading! 🚀